UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 1
PRIMER SEMESTRE 2026
PROYECTO 1

Andy Jefferson González Fuentes 202231338

MANUAL TÉCNICO

PKM Forms es una aplicación móvil para Android que permite diseñar, analizar y visualizar formularios interactivos mediante un lenguaje de programación propio con extensión .form. El usuario escribe el código en el editor integrado y el sistema lo procesa para mostrar el formulario renderizado en pantalla. El sistema está compuesto por dos grandes módulos: una aplicación Android (PkmForms2) que contiene el editor, el analizador lexico-sintactico y el visualizador, y una API REST en Java Spring Boot (pkm-api) que actúa como servidor central para almacenar y distribuir los formularios compilados.

Tecnologías utilizadas:

Componente	Tecnología	Versión
Aplicación móvil	Android / Kotlin + Jetpack Compose	Android SDK 34
Lexer / Parser	JFlex + CUP (Java)	JFlex 1.8 / CUP 0.11b
API REST	Java Spring Boot	Spring Boot 3.x
Build system	Gradle (Android) / Maven (API)	Gradle 8.13
API externa	PokeAPI (https://pokeapi.co)	v2

Arquitectura del sistema:

Capas de la aplicación:

Capa	Paquete	Responsabilidad
UI	ui.theme / ui.theme.components	Pantallas Compose, editor, visualizador, diálogos
Analizador	analyzer	PkmLexer, PkmParser, PkmExporter, PkmImporter
Nodos AST	analyzer.generated / analyzer.code	Nodos del árbol sintáctico generados por CUP
Modelo	analyzer.model	FormElement, StyleData, ErrorToken
API	api	ApiService, ApiConfig, PokemonApi

El sistema sigue una arquitectura cliente-servidor. La aplicación Android actúa como cliente y se comunica con la API REST mediante HTTP. Internamente, la aplicación está organizada en capas bien definidas.

Flujo del sistema:
El flujo completo desde que el usuario escribe código hasta que el formulario se visualiza es el siguiente:

- El usuario escribe código .form en el editor (EditorScreen).
- PkmLexer.normalizarColoresAngulares() preprocesa el código convirtiendo la notación <r,g,b> a (r,g,b).
- LexerPRO1 (generado por JFlex) tokeniza el código preprocesado.
- ParserPRO1 (generado por CUP) construye el AST validando la gramática LALR.
- PkmParser interpreta el AST, evalúa expresiones y estructuras de control, y produce una lista de FormElement.
- La UI de Jetpack Compose renderiza los FormElement como componentes visuales interactivos.

Lenguaje PKM FORMS:
PKM Forms es un lenguaje de dominio específico (DSL) declarativo-imperativo. Declarativo en la definición de elementos de formulario e imperativo en el uso de variables y estructuras de control.

Tipo	Declaración	Descripción
number	number x = 10	Número de punto flotante (Double internamente)
string	string s = "Hola"	Cadena de texto con soporte de emojis @[...]
special	special q = OPEN_QUESTION [...]	Plantilla reutilizable de pregunta con comodines ?

Elemento	Descripción
OPEN_QUESTION	Pregunta de respuesta abierta (campo de texto libre)
SELECT_QUESTION	Pregunta de selección simple con una respuesta correcta (maximo 5 opciones)
MULTIPLE_QUESTION	Pregunta de selección múltiple con varias respuestas correctas
DROP_QUESTION	Pregunta desplegable; soporta who_is_that_pokemon() para cargar opciones desde PokeAPI
TEXT	Elemento de texto estático con estilos personalizados
SECTION	Contenedor de elementos con orientación VERTICAL u HORIZONTAL
TABLE	Tabla de filas y columnas; cada celda puede contener cualquier elemento

Categoría	Operadores
Aritméticos	+ - * / % ^
Comparación	> >= < <= == !!
Lógicos	&& (AND) || (OR) ~ (NOT)
Concatenación	+ (entre strings o mixto, ej: "Valor: " + variable)

Los estilos se definen dentro del bloque styles [...] de cada elemento. Los colores soportan cuatro formatos:

- #RRGGBB — Hexadecimal. Ej: #1F4E79
- (R, G, B) — RGB con paréntesis. Ej: (30, 130, 80)
- <R, G, B> — RGB con ángulos (equivalente al anterior). Ej: <120, 80, 200>
- hsl(H, S, L) — HSL. Ej: hsl(210, 80, 45)
- Nombre predefinido: RED, BLUE, GREEN, YELLOW, BLACK, WHITE, PURPLE, SKY

Estructura del Analizador:

El analizador esta compuesto por tres etapas en cadena:

Etapa	Clase / Herramienta	Descripción
Preprocesamiento	PkmLexer.normalizarColoresAngulares()	Convierte <r,g,b> a (r,g,b) sin alterar strings ni comentarios
Analisis Lexico	LexerPRO1.java (JFlex)	Tokeniza el código; reporta errores léxicos con linea y columna
Analisis Sintactico	ParserPRO1.java (CUP)	Construye el AST mediante gramática LALR; reporta errores sintácticos
Interpretación	PkmParser.kt	Recorre el AST, evalúa expresiones y produce List<FormElement>

El lexer reconoce los siguientes grupos de tokens:

- **Palabras reservadas:** SECTION, TABLE, TEXT, OPEN_QUESTION, DROP_QUESTION, SELECT_QUESTION, MULTIPLE_QUESTION, IF, ELSE, WHILE, DO, FOR, IN, number, string, special, VERTICAL, HORIZONTAL, DRAW, WIDTH, HEIGHT y otros atributos.
- **Tipos de color:** COLOR_HEX (#RRGGBB), COLOR_RGB (r,g,b), COLOR_HSL, COLOR_PREDEFINIDO (RED, BLUE, etc.).
- **Literales:** NUMERO (Double), CADENA (String con emojis resueltos), IDENTIFICADOR.
- **Operadores:** MAS, MENOS, POR, DIV, POT, MOD, MAYOR, MENOR, IGUAL_IGUAL, DIFERENTE, AND, OR, NOT.
- **Delimitadores:** CORCH_ABRE/CIERRA, LLAVE_ABRE/CIERRA, PAREN_ABRE/CIERRA, COMA, DOS_PUNTOS, PUNTO_COMA, PUNTO, DOTDOT, COMODIN (?).
- **Especiales:** WHO_IS_THAT_POKEMON, NUMBER_CONST, HSL_FUNC.

Mensajes de Error:

El parser genera mensajes descriptivos según el contexto del error. Los más comunes son:

Situacion	Mensaje generado
Falta ; en FOR clasico	Se encontro '<' donde se esperaba ';'. El FOR clasico separa sus partes con punto y coma.
Falta , entre atributos	Se esperaba ',' antes de '<atributo>'. Los atributos deben separarse con coma.
Variable no declarada	Asignación inesperada. Verifique que la variable este declarada.
Fin de archivo inesperado	Fin de archivo inesperado. Verifique que todos los '[', '{' y '(' esten cerrados.
draw() con args incorrectos	draw() de 'nombre' recibio N argumento(s) pero la variable tiene M comodín(es).

Clases Principales del Proyecto:

FormElement

Clase sellada que representa cualquier elemento visual del formulario interpretado. Todas las subclases son data classes inmutables.

Subclase	Campos clave
OpenQuestion	label, width, height, style: StyleData
DropQuestion	label, options: List<String>, correct: Int, width, height, style
SelectQuestion	label, options: List<String>, correct: Int, width, height, style
MultipleQuestion	label, options: List<String>, correct: List<Int>, width, height, style
TextElement	content: String, width, height, style
Section	orientation, width, height, pointX, pointY, elements: List<FormElement>, style
Table	rows: List<List<FormElement?>>, width, height, pointX, pointY, style

StyleData

Data class que encapsula los atributos de estilo visual de cualquier elemento: color, backgroundColor, fontFamily, textSize, borderSize, borderType, borderColor.

ErrorToken

Data class que representa un error detectado. Contiene: lexeme (lexema involucrado), line, column, type (ErrorType enum: LEXICO, SINTACTICO, SEMANTICO, ADVERTENCIA) y description.

Nodos AST

El parser CUP construye el AST usando estas clases Java:

Clase	Paquete	Descripcion
StyleNode	generated	Estilos en bruto del parser (Object para soportar comodines)
SpecialNode	generated	Definición de una variable special con sus comodines
OpenQuestionNode	generated	Nodo de OPEN_QUESTION del AST
DropQuestionNode	generated	Nodo de DROP_QUESTION del AST
SelectQuestionNode	generated	Nodo de SELECT_QUESTION del AST
MultipleQuestionNode	generated	Nodo de MULTIPLE_QUESTION del AST
TextNode	generated	Nodo de TEXT del AST; tiene contentExpr para expresiones dinámicas
SectionNode	generated	Nodo de SECTION con lista de elementos hijos
TableNode	generated	Nodo de TABLE con filas (ArrayList<ArrayList<Object>>)
IfNode	code	Nodo IF con lista de ramas (condición + cuerpo)
WhileNode	code	Nodo WHILE con condición y cuerpo
DoWhileNode	code	Nodo DO WHILE con condición y cuerpo
ForNode	code	Nodo FOR (rango o clásico). esRango=true para FOR (i in 1..3)

PkmLexer

Wrapper Kotlin sobre LexerPRO1. Responsabilidades:

- getErrors(code): Ejecuta el lexer y retorna la lista de errores léxicos como List<ErrorToken>.
- normalizarColoresAngulares(code) [companion object]: Preprocesa el codigo fuente reemplazando la sintaxis de color <r,g,b> por (r,g,b). Omite strings, comentarios de bloque (/* */) y comentarios de linea ($). Función clave para compatibilidad con el parser sin necesidad de regenerar el LALR.

PkmParser

Motor de interpretación del AST. Responsabilidades principales:

- parse(code): Método principal suspend. Invoca lexer + parser, recorre ParserPRO1.resultado en dos pasadas (variables/speciales y luego conversión a FormElement) y retorna ParseResult.

- evaluarExpr(v): Evalua recursivamente cualquier expresión del AST (aritmética, lógica, comparación, concatenación).
- interpretarIf/While/DoWhile/For: Ejecutan las estructuras de control actualizando el mapa de simbolos e interpretando el cuerpo para producir elementos visuales.
- convertirSpecial(node, args): Instancia una variable special sustituyendo los comodines con los argumentos de draw() en orden.
- resolverColorConArgs(v, args, idx): Resuelve colores del formato rgb_expr:r:g:b con soporte de comodines y expresiones aritmeticas.

PkmExporter

Objeto singleton que serializa una List<FormElement> al formato binario .pkm (texto estructurado con etiquetas XML-like). Incluye un bloque de metadatos ### ... ### con autor, fecha, hora y estadísticas del formulario (total de secciones y preguntas por tipo).

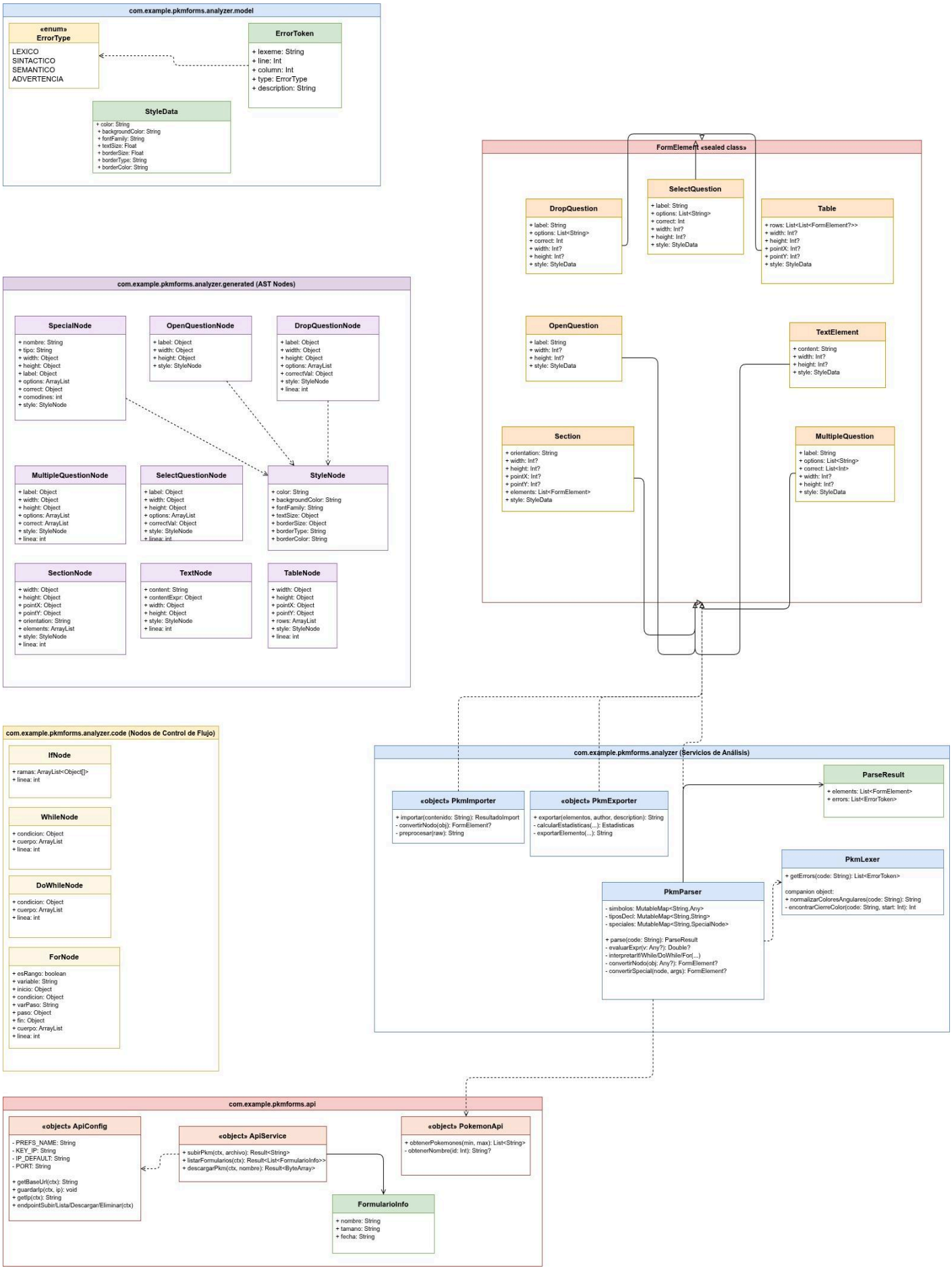
PkmImporter

Objeto singleton que deserializar un archivo .pkm de vuelta a List<FormElement>. Preprocesa el contenido (elimina metadatos, normaliza decimales y colores) antes de pasarlo al LexerPKM/ParserPKM secundario que entiende el formato .pkm.

API:

Clase	Descripción
ApiConfig	Singleton que gestiona la URL base del servidor (IP + puerto 8080) usando SharedPreferences. Expone endpoints: /formularios/subir, /lista, /descargar.
ApiService	Singleton con corrutinas (Dispatchers.IO) para subir, listar y descargar archivos .pkm via HTTP multipart/form-data y JSON.
PokemonApi	Consume https://pokeapi.co/api/v2/pokemon/{id} para obtener nombres de Pokemon en el rango [min,max] definido en who_is_that_pokemon().
FormularioInfo	Data class con los campos nombre, tamaño y fecha retornados por el endpoint /lista del servidor.

Método	Endpoint	Descripción
POST	/formularios/subir	Recibe un archivo .pkm via multipart/form-data y lo guarda en disco.
GET	/formularios/lista	Retorna JSON con la lista de formularios: nombre, tamaño y fecha.
GET	/formularios/descargar/{n}	Descarga el archivo .pkm con el nombre especificado.


com.example.pkmforms.analyzer.model
«enum»
ErrorType
LEXICO
SINTACTICO
SEMANTICO
ADVERTENCIA
ErrorToken
+ lexeme: String
+ line: Int
+ column: Int
+ type: ErrorType
+ description: String
StyleData
+ color: String
+ backgroundColor: String
+ fontFamily: String
+ textSize: Float
+ borderSize: Float
+ borderType: String
+ borderColor: String
FormElement «sealed class»
DropQuestion
+ label: String
+ options: List<String>
+ correct: Int
+ width: Int?
+ height: Int?
+ style: StyleData
SelectQuestion
+ label: String
+ options: List<String>
+ correct: Int
+ width: Int?
+ height: Int?
+ style: StyleData
Table
+ rows: List<List<FormElement?>>
+ width: Int?
+ height: Int?
+ pointX: Int?
+ pointY: Int?
+ style: StyleData
OpenQuestion
+ label: String
+ width: Int?
+ height: Int?
+ style: StyleData
TextElement
+ content: String
+ width: Int?
+ height: Int?
+ style: StyleData
Section
+ orientation: String
+ width: Int?
+ height: Int?
+ pointX: Int?
+ pointY: Int?
+ elements: List<FormElement>
+ style: StyleData
MultipleQuestion
+ label: String
+ options: List<String>
+ correct: List<Int>
+ width: Int?
+ height: Int?
+ style: StyleData
com.example.pkmforms.analyzer.generated (AST Nodes)
SpecialNode
+ nombre: String
+ tipo: String
+ width: Object
+ height: Object
+ label: Object
+ options: ArrayList
+ correct: Object
+ comodines: int
+ style: StyleNode
OpenQuestionNode
+ label: Object
+ width: Object
+ height: Object
+ style: StyleNode
DropQuestionNode
+ label: Object
+ width: Object
+ height: Object
+ options: ArrayList
+ correctVal: Object
+ style: StyleNode
+ linea: int
MultipleQuestionNode
+ label: Object
+ width: Object
+ height: Object
+ options: ArrayList
+ correct: ArrayList
+ style: StyleNode
+ linea: int
SelectQuestionNode
+ label: Object
+ width: Object
+ height: Object
+ options: ArrayList
+ correctVal: Object
+ style: StyleNode
+ linea: int
StyleNode
+ color: String
+ backgroundColor: String
+ fontFamily: String
+ textSize: Object
+ borderSize: Object
+ borderType: String
+ borderColor: String
SectionNode
+ width: Object
+ height: Object
+ pointX: Object
+ pointY: Object
+ orientation: String
+ elements: ArrayList
+ style: StyleNode
+ linea: int
TextNode
+ content: String
+ contentExpr: Object
+ width: Object
+ height: Object
+ style: StyleNode
+ linea: int
TableNode
+ width: Object
+ height: Object
+ pointX: Object
+ pointY: Object
+ rows: ArrayList
+ style: StyleNode
+ linea: int
com.example.pkmforms.analyzer.code (Nodos de Control de Flujo)
IfNode
+ ramas: ArrayList<Object[]>
+ linea: int
WhileNode
+ condicion: Object
+ cuerpo: ArrayList
+ linea: int
DoWhileNode
+ condicion: Object
+ cuerpo: ArrayList
+ linea: int
ForNode
+ esRango: boolean
+ variable: String
+ inicio: Object
+ condicion: Object
+ varPaso: String
+ paso: Object
+ fin: Object
+ cuerpo: ArrayList
+ linea: int
com.example.pkmforms.analyzer (Servicios de Análisis)
«object» PkmImporter
+ importar(contenido: String): ResultadoImport
- convertirNodo(obj): FormElement?
- preprocesar(raw): String
«object» PkmExporter
+ exportar(elementos, author, description): String
- calcularEstadisticas(...): Estadisticas
- exportarElemento(...): String
ParseResult
+ elements: List<FormElement>
+ errors: List<ErrorToken>
PkmLexer
+ getErrors(code: String): List<ErrorToken>
companion object:
+ normalizarColoresAngulares(code: String): String
- encontrarCierreColor(code: String, start: Int): Int
PkmParser
- simbolos: MutableMap<String,Any>
- tiposDecl: MutableMap<String,String>
- speciales: MutableMap<String,SpecialNode>
+ parse(code: String): ParseResult
- evaluarExpr(v: Any?): Double?
- interpretarIf/While/DoWhile/For(...)
- convertirNodo(obj: Any?): FormElement?
- convertirSpecial(node, args): FormElement?
com.example.pkmforms.api
«object» ApiConfig
- PREFS_NAME: String
- KEY_IP: String
- IP_DEFAULT: String
- PORT: String
+ getBaseUrl(ctx): String
+ guardarIp(ctx, ip): void
+ getIp(ctx): String
+ endpointSubir/Lista/Descargar/Eliminar(ctx)
«object» ApiService
+ subirPkm(ctx, archivo): Result<String>
+ listarFormularios(ctx): Result<List<FormularioInfo>>
+ descargarPkm(ctx, nombre): Result<ByteArray>
«object» PokemonApi
+ obtenerPokemones(min, max): List<String>
- obtenerNombre(id: Int): String?
FormularioInfo
+ nombre: String
+ tamano: String
+ fecha: String